

2025 종합설계

CNN Convolution 연산을 위한 준-범용 FPGA 가속기 설계 및 구현

중앙대학교 전자전기공학부

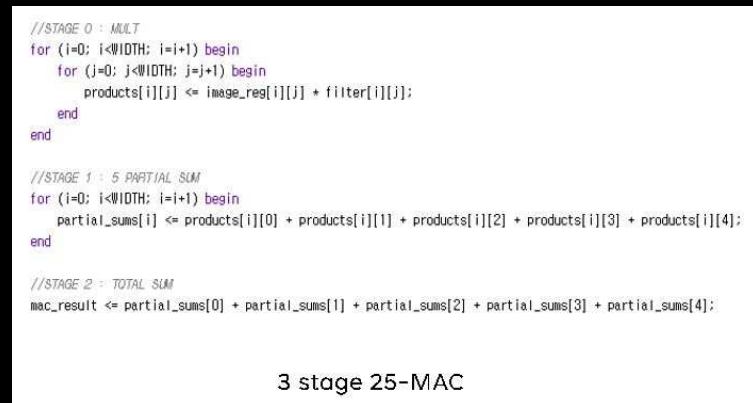
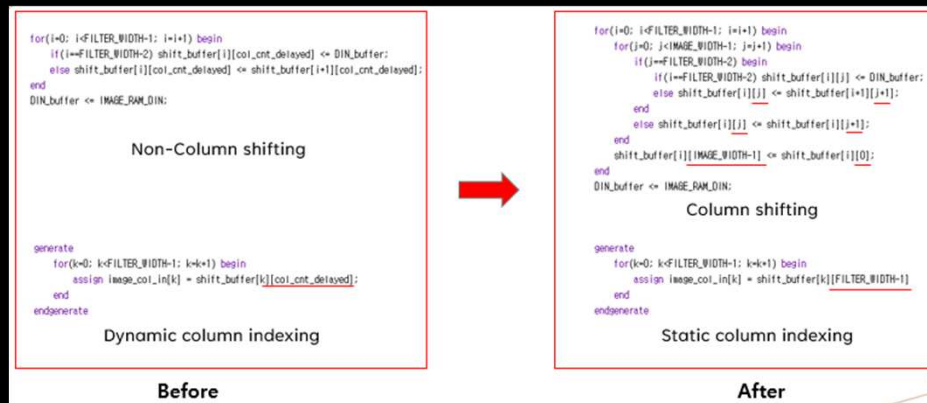
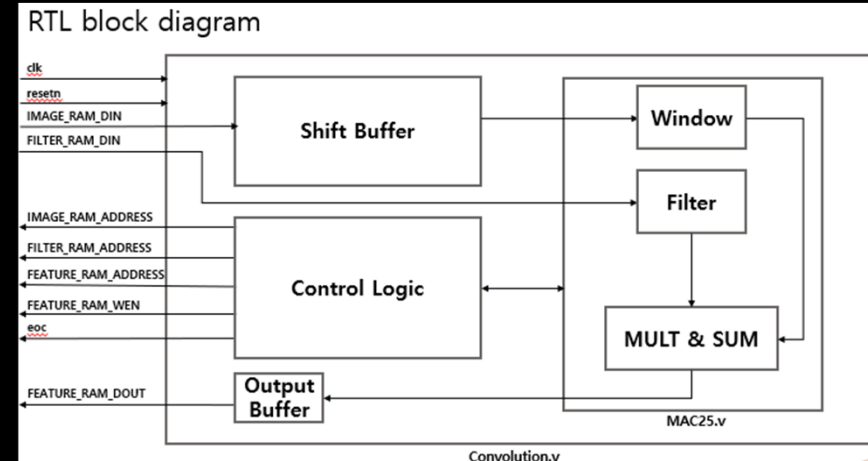
➤ 목차

- 0. 사전 계획
- 1. 서론
- 2. 이론적 배경
- 3. 하드웨어 설계
- 4. 실험 및 검증
- 5. 결론

➤ 0.1 사전 계획 – MNIST 추론 컨볼루션 가속기 구조

✓ 주요 구성 요소

- Shift Buffer : 입력 이미지 데이터를 받아 임시 저장
- Window/Filter : 컨볼루션 연산에 필요한 3x3 윈도우와 필터
- MULT & SUM : 곱셈 및 누산
- Control Logic : 주소, 타이밍, 상태 제어



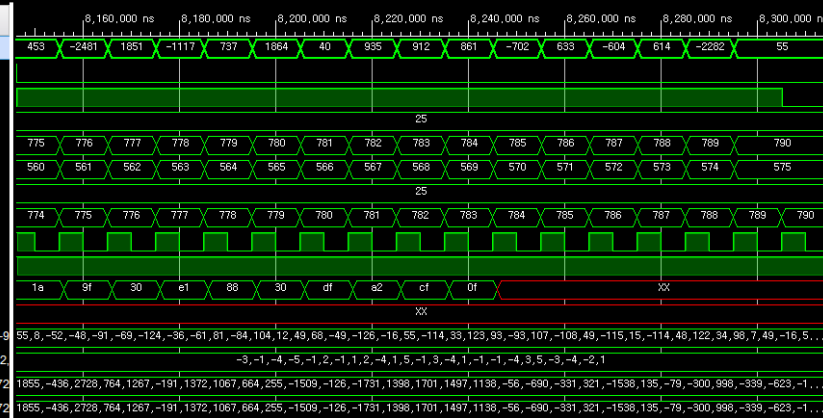
가속 아이디어 기반으로 RTL 설계 및 Testbench 시뮬레이션까지 진행

➤ 0.2 사전 계획 – RTL 설계 및 시뮬레이션

```
initial begin
  for (idx = 0; idx < IFMAP_DEPTH; idx = idx+1) begin
    din_memory[idx] = $urandom_range(255, 0);
  end
  for (idx = 0; idx < WEIGHT_DEPTH; idx = idx+1) begin
    win_memory[idx] = $urandom_range(5, -5);
  end
  for (idx = 0; idx < OFMAP_DEPTH; idx = idx+1) begin
    dout_memory[idx] = 0;
  end

  // ref_out
  idx = 0;
  for (i = 0; i <= input_height-kernel_size; i = i+stride) begin
    for (j = 0; j <= input_width-kernel_size; j = j+stride) begin
      sum = 0;
      for (m = 0; m < kernel_size; m++) begin
        for (n = 0; n < kernel_size; n++) begin
          ifmap_val = din_memory[(i+m)*input_width + (j+n)];
          weight_val = win_memory[m*kernel_size + n];
          sum += ifmap_val * weight_val;
        end
      end
      ref_out[idx] = sum;
      idx=idx+1;
    end
  end
end
```

Name	Value
dout[15:0]	55
eoc	1
o_wea	0
w_addr[9:0]	25
i_addr[9:0]	790
o_addr[9:0]	575
w_addr_buf[9:0]	25
i_addr_buf[9:0]	790
clk	1
rst	1
din[7:0]	XX
win[7:0]	XX
din_memory[0:783][7:0]	55,8,-52,-48,-91,-69,-124,-36,-61,81,-84,104,12,49,68,-49,-126,-16,55,-114,33,123,93,-93,107,-108,49,-115,15,-114,48,122,34,98,7,49,-16,5...
win_memory[0:24][7:0]	-3,-1,-4,-5,-1,2,-1,1,2,-4,1,5,-1,3,-4,1,-1,-1,-4,3,5,-3,-4,-2,1
dout_memor..575[15:0]	1855,-436,2728,764,1267,-191,1372,1067,664,255,-1509,-126,-1751,1398,1701,1497,1138,-56,-690,-331,321,-1538,135,-79,-300,998,-339,-623,-1...
ref_out[0:575][15:0]	1855,-436,2728,764,1267,-191,1372,1067,664,255,-1509,-126,-1751,1398,1701,1497,1138,-56,-690,-331,321,-1538,135,-79,-300,998,-339,-623,-1...



```
[570] Correct! Calculated : -702, Golden : -702
#####
[571] Correct! Calculated : 633, Golden : 633
#####
[572] Correct! Calculated : -604, Golden : -604
#####
[573] Correct! Calculated : 614, Golden : 614
#####
[574] Correct! Calculated : -2282, Golden : -2282
#####
[575] Correct! Calculated : 55, Golden : 55
```

But, 정해진 규격의 MNIST 이미지 추론, 그 중에서도 오직 첫번째 컨볼루션 레이어 연산에만 특화된 모듈

→ Task가 너무 쉽고 간단하다는 교수님의 피드백으로, 새로운 주제 선정하기로 결정

➤ 1.1 연구 배경 및 필요성

✓ 연구 배경

딥러닝 발전과 함께 CNN이 핵심 알고리즘으로 부상했으나, '컨볼루션' 연산의 막대한 연산량이 문제점으로 대두

- CPU : 병렬성이 낮아 실시간 처리에 비효율적
 - GPU/TPU : 일반적으로 고가, 고전력으로 인한 임베디드 환경 부적합
- FPGA는 병렬 연산 구조 및 재구성 가능성 덕분에 소형 CNN 가속 프로젝트에 적합

✓ 기존 연구의 한계

기존 학부 수준의 CNN 가속기는 특정 모델에 종속되는 경우가 많음

→ CNN 모델 자체, 혹은 컨볼루션 레이어의 하이퍼 파라미터 변경 시 재설계 필요

✓ 본 연구의 필요성

- 다양한 입력 크기, 채널 수, 패딩, 스트라이드에 대응하는 준-범용 컨볼루션 연산 가속 가능
- Zynq SoC 기반 HW/SW 통합 구조로 실시간 CNN 추론 실현 가능

➤ 1.2 연구 목적 및 목표

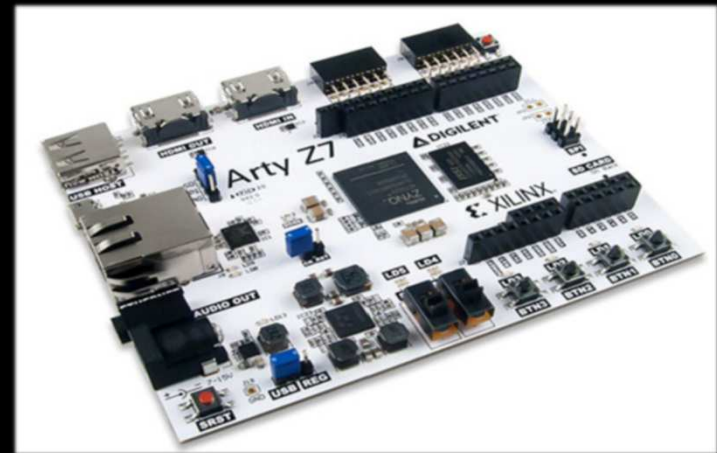
✓ 목적 및 목표

Zynq-7000 SoC(Arty Z7-20) 기반에서 CNN 컨볼루션 연산을 효율적으로 가속하기 위한,
준-범용 컨볼루션 가속 IP 설계 및 PYNQ 기반 HW/SW 통합 성능 검증

✓ 핵심 구조

Zynq SoC = PS(ARM Cortex-A9 CPU) + PL(FPGA)

- PS : 제어 및 데이터 전송 / PL : 컨볼루션 연산 가속 수행
- AXI-Lite + AXI4-Stream 프로토콜 기반 SoC 하이브리드 추론 구현



➤ 2.1 CNN 컨볼루션 연산 원리

✓ CNN의 컨볼루션 연산

CNN은 입력 이미지로부터 특징을 추출하기 위해 컨볼루션 연산을 반복적으로 수행

→ 입력 이미지의 특정 영역에 필터를 적용해 곱셈-누산 연산을 수행하며 출력 피쳐맵을 생성

$$O_{x,y,k} = \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} \sum_{c=0}^{C-1} I_{(x+i),(y+j),c} \times W_{i,j,c,k} + B_k$$

✓ 문제점

입출력 채널 수가 많아질수록 연산량이 폭발적으로 증가 → 실시간 처리 어려움

✓ 본 연구의 접근

컨볼루션 연산의 빠른 병렬 처리 및 자원 효율 극대화를 위한 방안

- Shift Buffer를 이용한 입력 데이터 재사용
- 16개의 병렬 MAC9 유닛을 통한 연산 병렬화 및 파이프라인 처리
- FIFO 누적 구조를 통한 효율적 내부 메모리 관리 등

➤ 2.2 정수형 양자화 이론

✓ 문제점

딥러닝 모델은 일반적으로 부동소수점 연산

→ 하지만 FPGA에서 FP32 연산을 구현하면 DSP 자원과 LUT, FF 소모량 증가 및 연산 지연이 증가

✓ 해결 방법 - 정수형 양자화(Quantization)

실수형 데이터 CNN의 입력, 가중치, 바이어스를 각각 정수형으로 스케일링(INT8/INT32) 하여 연산 효율 증대

✓ 양자화 및 역양자화 과정

$$S_x = \frac{\max(|x_f|)}{2^7 - 1} = \frac{\max(|x_f|)}{127}$$

$$S_w = \frac{\max(|W_f|)}{2^7 - 1} = \frac{\max(|W_f|)}{127}$$

$$S_{out} = S_x \times S_w$$

$$x_q = \text{round} \left(\frac{x_f}{S_x} \right)$$

$$W_q = \text{round} \left(\frac{W_f}{S_w} \right)$$

$$B_q = \text{round} \left(\frac{B_f}{S_{out}} \right)$$

$$Y_q = \text{Conv}(x_q, W_q) + B_q$$

$$Y_f \approx S_{out} \times Y_q$$

✓ 효과

연산 자원 절감 : DSP, LUT, FF 사용량 감소

연산 속도 향상 : 연산 지연 시간 단축

➤ 2.2.1 정수형 양자화(Quantization) 검증

✓ 검증 목적

실제 하드웨어 설계 전 Python 기반의 시뮬레이션을 통해 양자화(Quantization) 및 데이터 흐름을 선행 검증하고, 정확도와 구현 타당성을 사전에 확보

✓ 검증 방법

① Custom Model 설계 및 학습

자체 설계한 CNN 모델 구축 및 FP32 기반 사전 학습(Pre-training)

② 가중치 추출 및 무결성 검증 (Weight Verification)

학습된 가중치(Weight) 데이터 추출 및 출력을 통해 저장/이동 가능 여부 확인

```
tensor([[[[ 2.0062e-01,  3.7847e-02, -1.3315e-01,  1.6217e-01,  1.3460e-01],
          [-4.2140e-02, -1.2622e-01,  7.6704e-05,  7.5088e-02, -4.0853e-02],
          [ 1.6773e-01,  1.4150e-01,  1.7949e-01, -1.2657e-03, -3.5551e-02],
          [ 7.9252e-02,  1.4399e-01, -3.6586e-02, -1.2157e-02, -1.1497e-01],
          [-2.9778e-01, -2.6868e-01, -2.7164e-01, -4.2871e-02, -2.4176e-01]]]])

[[[-1.9415e-01, -1.3360e-02, -2.4732e-01, -2.4844e-01, -2.0394e-01],
  [ 1.2005e-01, -7.2127e-02, -7.7575e-04, -1.0917e-01, -2.3973e-01],
  [ 1.7857e-02,  3.6709e-02,  5.5423e-02,  8.1934e-03, -7.2865e-02],
  [ 1.1500e-01, -2.6902e-02,  7.6380e-02,  3.0852e-01,  2.4223e-01],
  [ 7.8273e-04,  9.9545e-02,  3.0567e-02,  6.2425e-02, -8.9706e-02]]]]
```

➤ 2.2.1 정수형 양자화(Quantization) 검증

✓ 검증 방법

③ Hook & Observer 기반 PTQ 적용

Hook을 통해 Layer별 Observer 적용, 데이터 분포 측정

측정값 기반의 Symmetric Quantization 수행 및 정수 변환

```
[Hook] 훅(Hook) 최초 실행됨! (첫부분 테스트)  
가로챤 레이어: Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
[저장 완료] 'vgg16_test_vectors_BACK.pt' 파일 생성됨.
```

④ 정확도 검증 및 유효성 판단

정수 연산 시뮬레이션 결과와 원본 비교

정확도 하락이 미미함을 확인하여 설계 타당성 확보

```
[Epoch 01] train_loss=0.4368 train_acc=91.01%  
[Epoch 02] train_loss=0.0969 train_acc=97.41%  
[Epoch 03] train_loss=0.0700 train_acc=97.97%  
[Epoch 04] train_loss=0.0570 train_acc=98.32%  
[Epoch 05] train_loss=0.0504 train_acc=98.51%  
[Epoch 06] train_loss=0.0439 train_acc=98.72%  
[Epoch 07] train_loss=0.0399 train_acc=98.82%  
[Epoch 08] train_loss=0.0360 train_acc=98.97%  
[Epoch 09] train_loss=0.0348 train_acc=98.95%  
[Epoch 10] train_loss=0.0321 train_acc=99.02%  
[FINAL TEST] loss=0.0386 acc=98.72%
```

➤ 2.3 Zynq SoC 및 시스템 요구사항

✓ Zynq SoC 개요

Zynq-7000 SoC는 하나의 칩 내에 Processing System(PS) 과 Programmable Logic(PL) 이 통합된 SoC 구조

✓ AXI 인터페이스

PS-PL 간 데이터 통신에 활용

제어	AXI-LITE	레지스터 제어 및 파라미터 설정
데이터 전송	AXI4-Stream	입출력 데이터의 스트리밍 전송
메모리 접근	AXI4 Memory-Mapped (DMA)	DRAM ↔ IP 간 데이터 버퍼링

➤ 2.3 Zynq SoC 및 시스템 요구사항

✓ 시스템 요구사항 요약

① 기능적 요구사항

필터 크기 : 3×3 고정 (패딩 : 0~2)

입력 크기 : 최소 1×1 ~ 최대 32×32

입력 채널 수 : 무제한

출력 채널 수 : 최대 512

스트라이드 : 무제한

② 정확성 및 성능 요구사항

FPGA 연산 결과가 CPU 연산 결과와 일치해야 함

CPU 대비 수 배 이상 속도 향상 및 추론 정확도 손실 최소화

③ 확장성 및 재사용성

RTL 설계 시 parameter / localparam 활용 → 유지 보수 및 확장 용이

Vivado IP Packager를 통해 AXI 기반 독립된 IP 블록으로 구현

④ HW/SW 통합

PYNQ 프레임워크 기반 PS-PL 제어 환경 구현

⑤ 자원 효율성

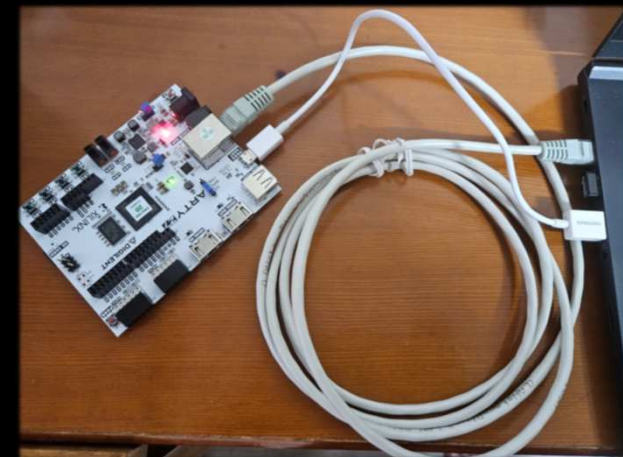
Arty Z7-20 보드의 제한된 LUT, FF, BRAM, DSP 자원 내 최적화

➤ 2.3 Zynq SoC 및 시스템 요구사항

✓ 개발 환경

FPGA 보드	Digilent Arty Z7-20 (Zynq-7020 SoC)
개발 도구	Vivado 2022.2
임베디드 환경	PYNQ 2.7 (Python 3.8)
실험 환경	Jupyter Notebook, PyTorch 1.8.1, Torchvision 0.9.1
인터페이스	AXI4-Lite (제어) / AXI4-Stream (데이터)

- ① PYNQ 공식 이미지를 SD 카드에 설치한 뒤, 해당 SD 카드를 Arty 보드에 장착 후 부팅하여 환경 구성
- ② PyTorch, Torchvision은 ARMv7l 아키텍처용 .whl 파일로 수동 설치
- ③ Vivado를 이용해 RTL 설계 → IP 패키징 → SoC 통합 수행
- ④ PYNQ가 제공하는 Jupyter Notebook에서 하드웨어 제어 및 검증 진행



➤ 2.3 Zynq SoC 및 시스템 요구사항

192.168.2.99:9090/terminals/2

jupyter

Logout

```
root@pynq:/# ls
bin boot dev etc home lib lib64 lost+found media mnt opt proc root run/sbin/srv/sys tmp usr var
root@pynq:/# cd home
root@pynq:/home# ls
jenkins xilinx
root@pynq:/home# cd xilinx/
root@pynq:/home/xilinx# ls
jupyter_notebooks pynq pynq_torch_env REVISION
root@pynq:/home/xilinx# cd jupyter_notebooks/
root@pynq:/home/xilinx/jupyter_notebooks# ls
base common loginbooks torch-1.8.1-cp38-cp38-linux_armv7l.whl 'Welcome to Pynq.ipynb'
cnn getting_started pynq_peripherals torchvision-0.9.1-cp38-cp38-linux_armv7l.whl
root@pynq:/home/xilinx/jupyter_notebooks#
```

➤ 2.3 Zynq SoC 및 시스템 요구사항



The image shows the JupyterLab file browser interface. At the top, there's a 'jupyter' logo and 'Quit' and 'Logout' buttons. Below that are tabs for 'Files', 'Running', and 'Clusters'. A message says 'Select items to perform actions on them.' with 'Upload', 'New', and a refresh icon. The main area shows a directory listing for the root directory '/'. The listing includes folders like 'base', 'cnn', 'common', 'getting_started', 'logictools', and 'pynq_peripherals', and files like 'Welcome to Pynq.ipynb', 'torch-1.8.1-cp38-cp38-linux_armv7l.whl', and 'torchvision-0.9.1-cp38-cp38-linux_armv7l.whl'. Each item has a checkbox, a name, a last modified date, and a file size.

	Name	Last Modified	File size
☐	/		
☐	base	4년 전	
☐	cnn	5월 전	
☐	common	4년 전	
☐	getting_started	4년 전	
☐	logictools	4년 전	
☐	pynq_peripherals	4년 전	
☐	Welcome to Pynq.ipynb	4년 전	1.89 kB
☐	torch-1.8.1-cp38-cp38-linux_armv7l.whl	한 달 전	41.1 MB
☐	torchvision-0.9.1-cp38-cp38-linux_armv7l.whl	한 달 전	15.3 MB

➤ 3.2 데이터 흐름 및 동작 시퀀스

✓ 연산 절차

① 파라미터 설정 단계

Python(PYNQ)에서 AXI-Lite를 통해 하드웨어 내 레지스터 쓰기/읽기

주소 오프셋	파라미터	역할
0x00	IMAGE_WIDTH	입력 이미지의 크기
0x04	INPUT_CHANNEL	입력 채널 수
0x08	OUTPUT_CHANNEL	출력 채널 수
0x0C	STRIDE	스트라이드 값
0x10	PADDING	패딩 값
0x14	START	연산 시작 트리거
0x18	EOC_STATUS	연산 종료 상태

② 입력 데이터 전송 단계

PS → DMA → PL 경로로 AXI4-Stream 데이터 전송

입력 이미지는 Shift Buffer에 저장, 필터 및 바이어스는 BRAM에 저장

[데이터 전처리 요구 사항]

입력 : 입력채널 x 행 x 열

필터 : 입력채널 x 출력채널 x 행 x 열

바이어스 : 필터배열에 이어 붙여(concatenate) 전송

➤ 3.2 데이터 흐름 및 동작 시퀀스

✓ 연산 절차

③ 연산 단계

Shift Buffer와 MAC9 병렬 유닛을 통해 3×3 윈도우 구성 및 컨볼루션 연산 수행
다중 입력 채널의 경우 FIFO를 기반으로 누적하며 최종 출력 피쳐맵 생성
가변적인 입력 크기, 채널 수, 패딩, 스트라이드에 대해 자동 대응

④ 출력 데이터 전송 단계

연산 완료 시 AXI4-Stream으로 DRAM에 저장

출력 데이터 구조 : 행 x 열 x 출력채널 → Python에서 transpose 처리하여 사용

이후 CPU 연산 결과와 비교 검증, 또는 후속 레이어(ReLU, BN 등)로 전달하여 추론 계속 진행

➤ 3.3 하드웨어 모듈 설계

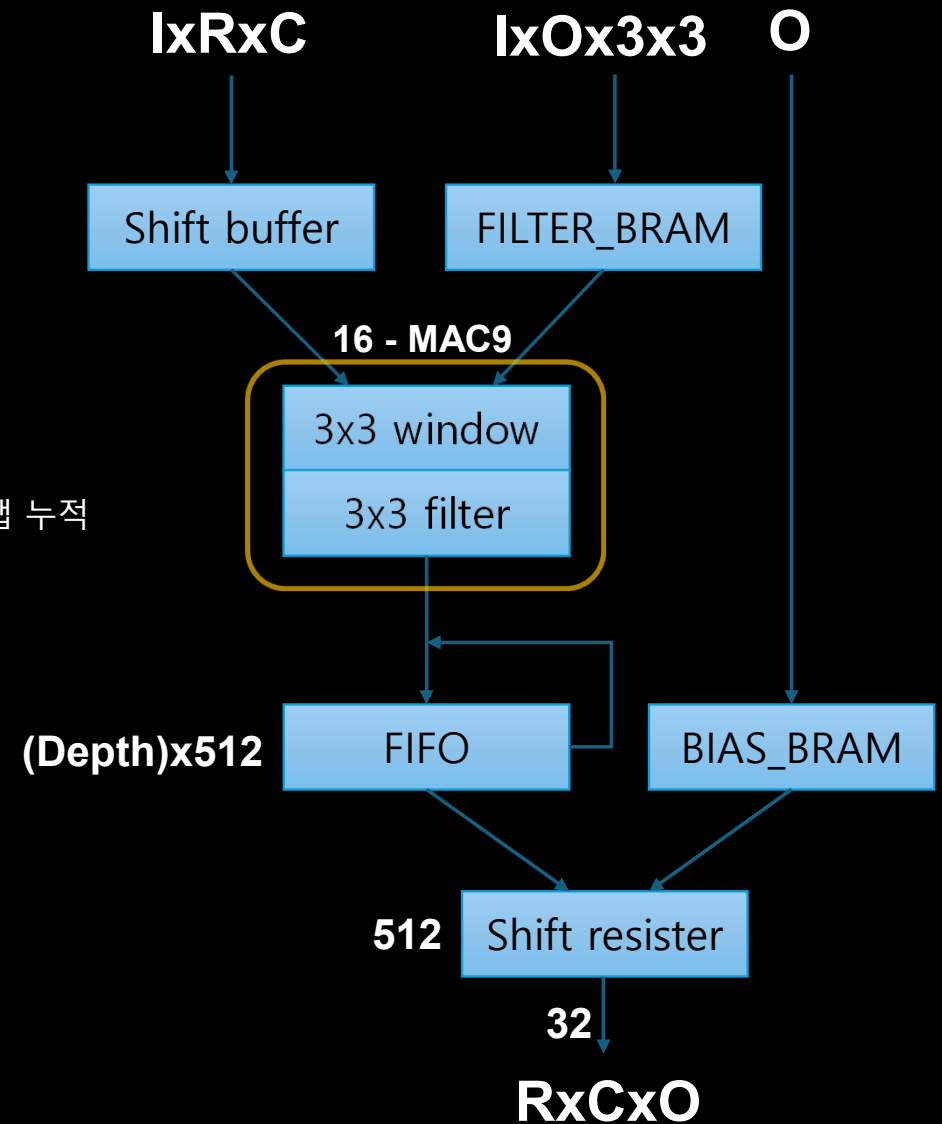
✓ Convolution IP의 주요 하위 구조

Shift Buffer : 입력 데이터 임시 저장

MAC9 병렬 유닛 : 3×3 윈도우 구성 및 컨볼루션 연산 수행

BRAM / FIFO 메모리 : BRAM → 필터, 바이어스 저장 / FIFO → 출력 피쳐맵 누적

FSM : 컨볼루션 연산 순서 관리, 제어



➤ 3.3.1 Shift Buffer

✓ 역할

입력 스트림 데이터를 임시 저장하며 재사용, 유효한 3×3 컨볼루션 윈도우를 형성할 수 있도록 돕는 핵심 아이디어

① 설계 의도

동적 인덱싱 방식은 MUX 합성과 함께 복잡 + 가변적인 입력, 패딩, 스트라이드 처리 어려움

→ 이미지 버퍼 자체를 시프트시키는 구조로 단순화

이미지 버퍼의 일부 고정된 위치의 레지스터만 MAC9 유닛들에 연결하여 연산에 사용할 3×3 윈도우를 구성

② 동작 원리

입력 스트림 → 1차 버퍼(DIN_buffer) 에 저장 및 두 개의 라인 버퍼와 함께 한 칸씩 시프트

특정 열의 3×1 스트립을 MAC9 유닛으로 전달

MAC9 유닛 내부에서 이 스트립을 다시 시프트하며 모아 최종 3×3 윈도우 완성

③ 효과

DRAM 접근 최소화, 데이터 재사용 최대화

다양한 입력 크기, 패딩, 스트라이드 조합에도 동일한 시프트 알고리즘 적용하며 대응 가능

➤ 3.3.2 MAC9 병렬 유닛

✓ 역할

Shift Buffer의 데이터로부터 3×3 윈도우를 형성하고 BRAM의 필터 데이터를 읽어 컨볼루션 연산을 수행하는 블록
각 유닛은 9개의 곱셈기와 누산기로 구성되어 한번에 1개 필터에 대한 컨볼루션 결과를 계산함

① 설계 의도

컨볼루션은 대규모 MAC 연산을 요구 → 단일 MAC 유닛만으로는 실시간 처리 어려움
16개의 MAC9 유닛 병렬 배치 → 한 클럭당 16개의 출력 채널 동시 계산 가능하도록 설계

② 동작 원리

FSM의 READ_FILTER 상태에서 필터 데이터가 AXI-Stream으로 유입 → FILTER_BRAM에 사전 저장
각 MAC9 유닛은 BRAM으로부터 서로 다른 필터를 로드하며, 구성해 놓은 3×3 윈도우와 컨볼루션 연산 수행
3단 파이프라인 구조 : 곱셈 → 행 단위 부분합 → 전체 합
파이프라인이 채워진 이후에는 매 클럭마다 16개의 결과 생성

③ 효과

병렬화로 인해 단일 유닛 대비 16배 연산 속도 향상
모듈화 구조 → 향후 보드 자원에 따라 병렬 유닛 수 확장 가능

➤ 3.3.3 BRAM 및 FIFO 메모리

✓ 역할

BRAM : 필터(Weight), 바이어스(Bias) 데이터 저장

FIFO : 각 입력 채널별 계산한 컨볼루션 결과를 순차적으로 저장 / 합산

① 설계 의도

다채널 컨볼루션에서는 각 입력 채널의 연산 결과를 임시 저장 한 뒤, 채널마다 누적해야 최종 출력 피쳐맵 완성
BRAM 기반 Random Access 방식은 가변적인 크기의 출력 피쳐맵을 호환하기 위해선 막대한 BRAM 사용량 필요
이를 해결하기 위해 FIFO 기반 순차 누적 구조 도입, 입력 채널별 연산 결과를 주소 접근 없이 임시 저장 및 누적
MAC9 유닛 연산 결과 단위 (16개×32bit)에 맞춰 FIFO의 폭은 고정, 깊이만 가변적으로 사용

② 동작 원리

다중 입력 채널 컨볼루션 연산을 단일 입력 채널 컨볼루션의 반복으로 단순화한다는 아이디어에 기반

본 설계에서 출력 피쳐맵은 입력 채널마다 (행×열×출력채널16개) 반복의 고정된 순서로 계산된다는 특성이 있어 FIFO가 매우 적합
최종적으로 바이어스와 더해지며 외부로 출력

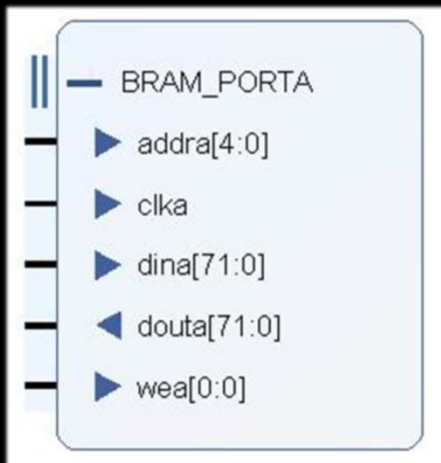
➤ 3.3.3 BRAM 및 FIFO 메모리

③ 효과

단순 RAM 저장방식 대비 BRAM 사용량 대폭 절감

주소 생성, 접근 로직 $X \rightarrow$ FSM 제어 단순화

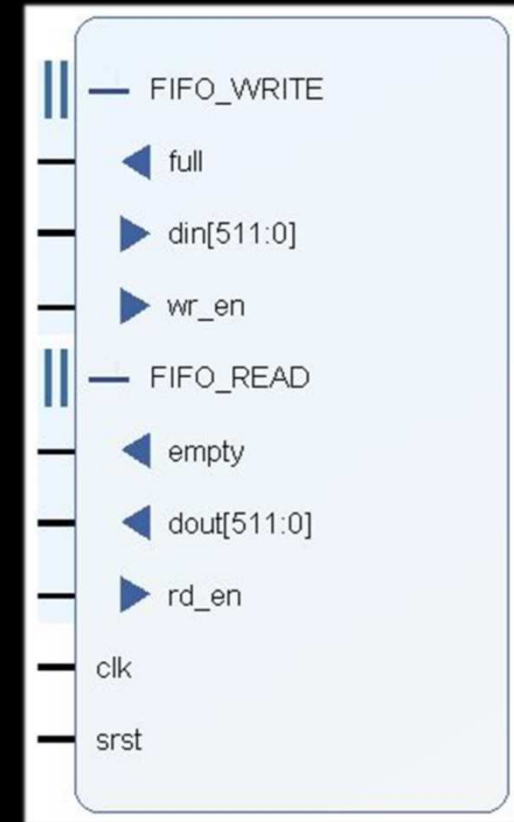
출력 채널 제한 확장 필요시 FIFO 깊이 조정만으로 해결 가능



FILTER_BRAM : 32x72



BIAS_BRAM : 32x32



FIFO : 8092x512

➤ 3.3.4 FSM 제어 및 동작

✓ 역할

컨볼루션 연산 전체 과정 및 각 하위 구조(Shift Buffer, MAC9, BRAM/FIFO 등)의 동작을 단계별로 제어

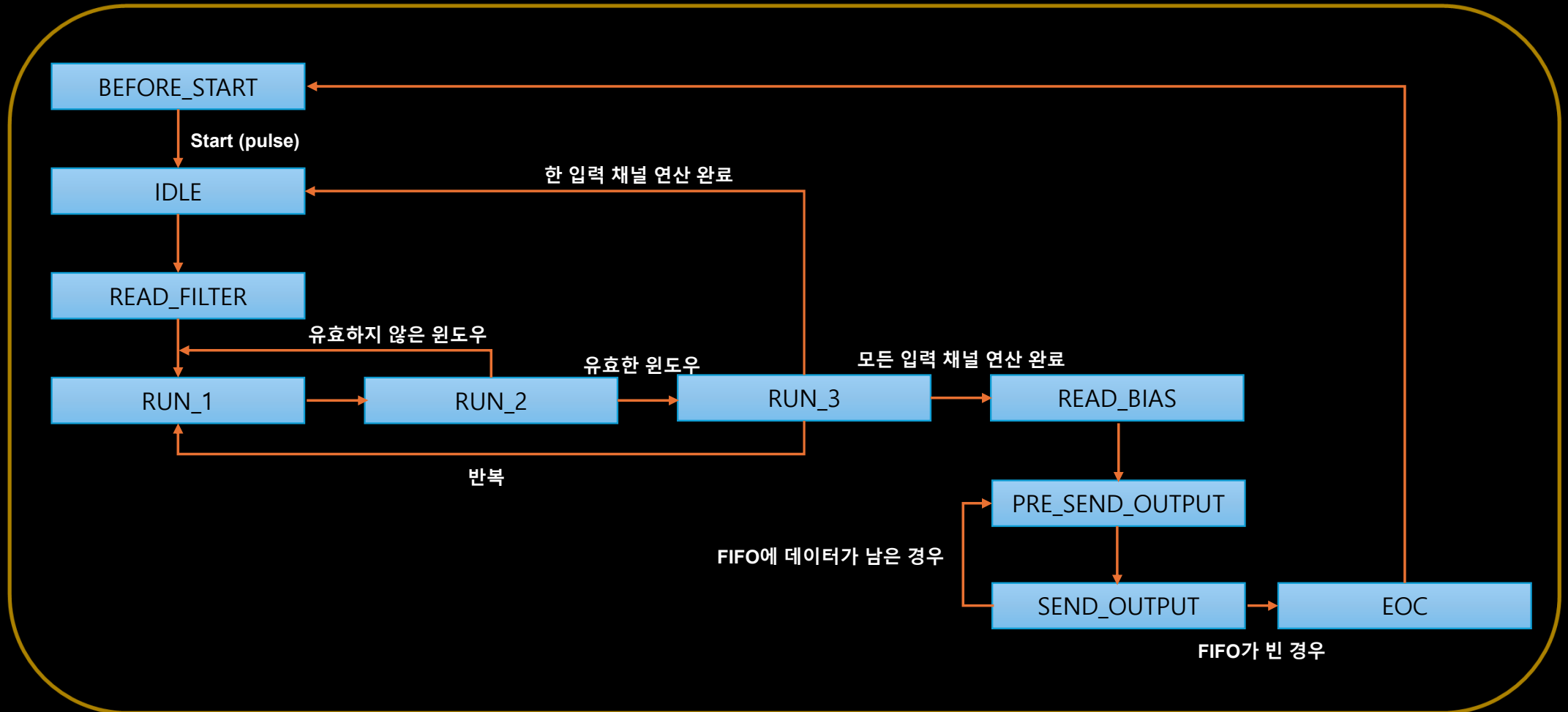
상태	설명
BEFORE_START	Start 신호 대기
IDLE	다음 입력채널 연산 시작 준비, 내부 레지스터 초기화
READ_FILTER	필터 데이터(한 채널) 수신 및 저장
RUN_1	현재 처리할 이미지가 유효 데이터인지/패딩 영역인지 판단
RUN_2	입력 이미지 임시 저장 및 시프팅, 스트라이드 판단
RUN_3	MAC9 병렬 컨볼루션 연산 수행, 결과 FIFO에 누적
READ_BIAS	바이어스 데이터 수신 및 저장
PRE_SEND_OUTPUT	(BIAS_BRAM 읽기 지연 타이밍을 맞추기 위한 1사이클 더미 상태)
SEND_OUTPUT	출력 피쳐맵에 바이어스를 더한 후 결과 송신
EOC	연산 완료를 알리며 BEFORE_START 상태로 복귀

✓ 특징

컨볼루션 연산의 입력-처리-출력까지의 모든 과정을 단계별로 분리

-> 연산 시작 트리거 이후의 모든 과정을 하드웨어 단독으로 자동 수행

➤ 3.3.4 FSM 제어 및 동작



➤ 3.3.5 가변 파라미터 대응 설계 아이디어

✓ 설계 아이디어

본 컨볼루션 IP는 가변적인 입력 크기, 패딩, 스트라이드, 입출력 채널 수에도 단일 하드웨어로 처리 가능

고정된 하드웨어 내에서 제어 신호(img_row_cnt, img_col_cnt, str_row_cnt, str_col_cnt 등)를 기반으로 다양한 규격의 컨볼루션 수행

① 입력 이미지 크기

라인버퍼 기반 Shift Buffer 구조 → $1 \times 1 \sim 32 \times 32$ 까지 유연하게 처리 가능

입력 받은 IMAGE_WIDTH / PADDING 값에 따라 Shift buffer의 데이터 시프팅 경로가 결정

→ 입력 이미지 크기와 관계없이 이후 과정은 동일함

실제 입력 이미지 데이터는 Shift buffer에 임시 저장

이미지 카운터(img_row_cnt, img_col_cnt)가 가상의 2차원 좌표 역할로 이미지 영역/패딩 영역 판별

→ 실제 입력 데이터 자체는 Shift buffer에 임시 저장되나, 몇 행 몇 열 데이터를 처리하는지는 카운터로 따로 관리

➤ 3.3.5 가변 파라미터 대응 설계 아이디어

② 패딩 및 스트라이드

패딩 : RUN_1 상태에서 현재 이미지 카운터 좌표가 패딩 범위(가장자리)면 0 패딩,
그렇지 않은 경우는 새로운 입력 데이터 스트리밍

스트라이드 : RUN_2 상태에서 $(str_row_cnt, str_col_cnt) \neq (0, 0)$ 일 때만 컨볼루션을 진행
만약 스트라이드값이 s 면, 스트라이드 카운터는 행, 열별로 각각 $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow s-1 \rightarrow 0$ 을 반복

➤ 3.3.5 가변 파라미터 대응 설계 아이디어

③ 입력 및 출력 채널

입력 채널 : 다중 입력 채널 컨볼루션 연산 = 단일 입력 채널 컨볼루션의 반복

각 입력 채널의 컨볼루션 결과는 FIFO를 통해 순차 누적

출력 채널(필터 수) : 16개의 병렬 MAC9 유닛이 한 번에 16개 필터씩 연산 수행

'used_filter' 카운터를 기반으로, 현재 처리한 필터 수를 추적하며 반복

OUTPUT_CHANNEL 만큼 출력 채널 처리 후 다음 상태로 전환

④ 정리 및 의의

이미지 좌표, 패딩 삽입 여부, 스트라이드 주기, 처리 채널 수를 판단하며 입력 한 하이퍼 파라미터의 컨볼루션을 정확히 수행

→ 하드웨어의 구조적 복잡도를 크게 늘리지 않고도 다양한 컨볼루션 연산에 대응 가능한 효율성

→ 특정 모델에 종속되지 않는 준-범용 CNN 가속

➤ 3.4 IP 생성 및 SoC 통합

✓ 설계 과정

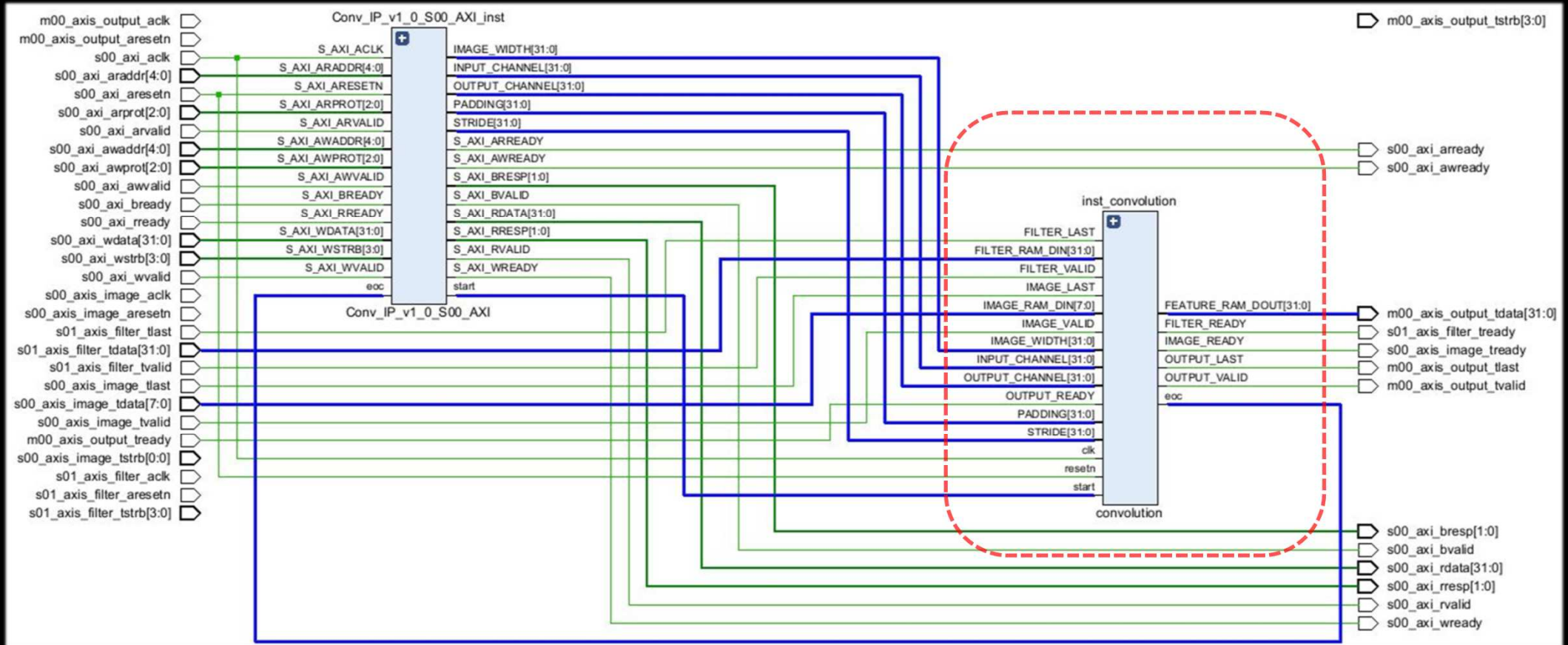
설계한 컨볼루션 가속 모듈을 Vivado에서 AXI 기반 사용자 정의 IP로 패키징 후, 블록 디자인을 통해 SoC로 구현

① IP 생성

설계한 모듈을 바탕으로 Vivado IP Packager를 이용해 AXI 인터페이스를 지원하는 독립적인 IP로 생성



➤ 3.4 IP 생성 및 SoC 통합



➤ 3.4 IP 생성 및 SoC 통합

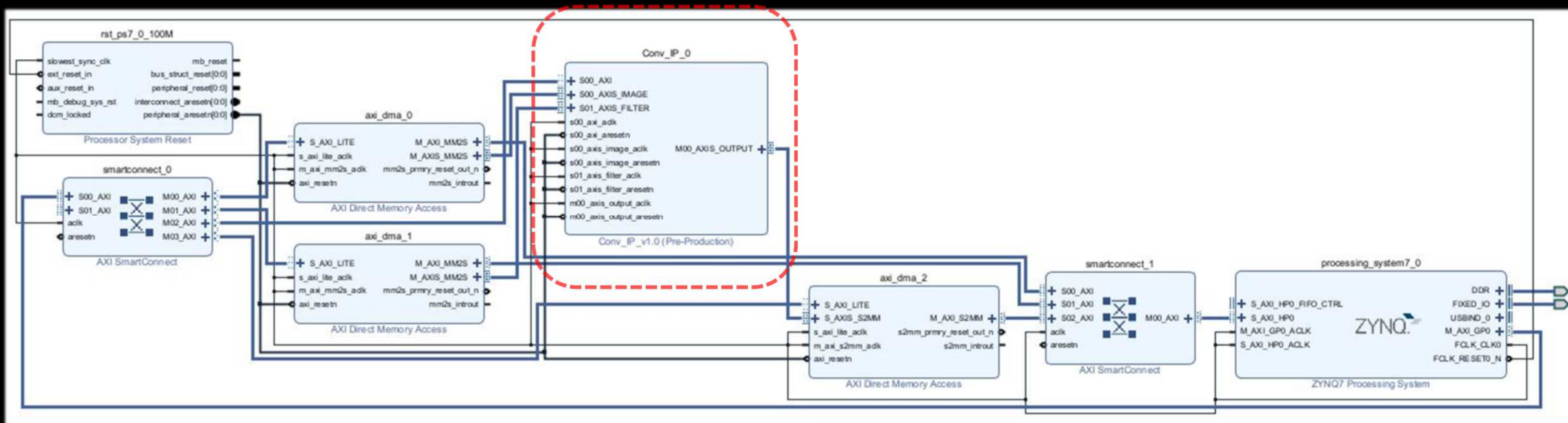
② 블록 디자인

패키징된 IP는 Vivado의 Block Design을 통해 Zynq7 Processing System, 3개의 AXI DMA, SmartConnect와 연결

Zynq7 PS : CPU (Python 코드 실행)

AXI DMA 3개 : 입력 / 필터, 바이어스 / 출력 데이터 스트리밍 용

SmartConnect : PS-DMA-IP 간 AXI 버스 연결



➤ 3.4 IP 생성 및 SoC 통합

③ 합성, 구현 및 비트스트림 생성

블록 디자인 후 Synthesis → Implementation을 거치며 타이밍 여유 및 리소스 사용량 확인

비트스트림 생성 후 보드에 업로드하여 PYNQ 환경에서 DMA 통신 및 IP 제어 동작 검증

→ 설계가 실제 SoC 시스템 내에서도 완전한 동작을 수행함을 확인

Hierarchy									
Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Block RAM Tile (140)	DSPs (220)	
design_1_wrapper	9399	10634	33	3673	8550	849	84	144	
design_1_i (design_1)	9399	10634	33	3673	8550	849	84	144	
axi_dma_0 (design_1_axi_dma_0_0)	482	694	0	202	448	34	1	0	
axi_dma_1 (design_1_axi_dma_1_0)	478	740	0	222	446	32	1	0	
axi_dma_2 (design_1_axi_dma_2_0)	899	1365	0	412	824	75	1	0	
Conv_IP_0 (design_1_Conv_IP_0_0)	3548	2915	32	1387	3548	0	81	144	
processing_system7_0 (design_1_processing_system7_0_0)	0	0	0	0	0	0	0	0	
rst_ps7_0_100M (design_1_rst_ps7_0_100M_0)	17	33	0	8	16	1	0	0	
smartconnect_0 (design_1_smartconnect_0_0)	1275	1246	1	490	1157	118	0	0	
smartconnect_1 (design_1_smartconnect_1_0)	2705	3641	0	1082	2116	589	0	0	

➤ 3.4 IP 생성 및 SoC 통합

③ 합성, 구현 및 비트스트림 생성

블록 디자인 후 Synthesis → Implementation을 거치며 타이밍 여유 및 리소스 사용량 확인

비트스트림 생성 후 보드에 업로드하여 PYNQ 환경에서 DMA 통신 및 IP 제어 동작 검증

→ 설계가 실제 SoC 시스템 내에서도 완전한 동작을 수행함을 확인



The image shows the JupyterLab file browser interface. The top bar includes the Jupyter logo, the text 'jupyter', and buttons for 'Quit' and 'Logout'. Below the top bar are tabs for 'Files', 'Running', and 'Clusters'. The 'Files' tab is active, showing a list of files and folders in the '/ cnn' directory. The list includes a folder '..', a folder 'data', a file 'cnn.ipynb' (marked as 'Running'), two 'Untitled.ipynb' files, a file '실험3. 실제 추론 적용 파인튜닝.ipynb', and four files highlighted by a red dashed box: 'cnn.bit', 'cnn.hwh', 'simple_vgg_trained.pth', and 'vgg16_best_params.pth'. The table also shows the last modified date and file size for each item.

Name	Last Modified	File size
..	몇 초 전	
data	15일 전	
cnn.ipynb	Running 14일 전	1.62 MB
Untitled.ipynb	한 달 전	17.7 kB
Untitled1.ipynb	한 달 전	595 B
실험3. 실제 추론 적용 파인튜닝.ipynb	14일 전	181 kB
cnn.bit	15일 전	4.05 MB
cnn.hwh	15일 전	502 kB
simple_vgg_trained.pth	15일 전	25.6 MB
vgg16_best_params.pth	14일 전	166 MB

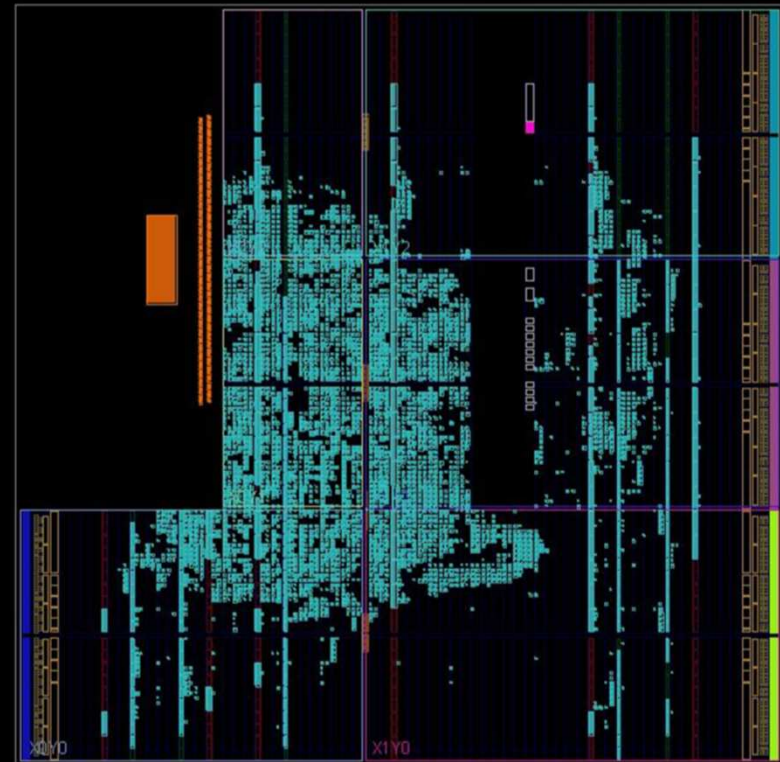
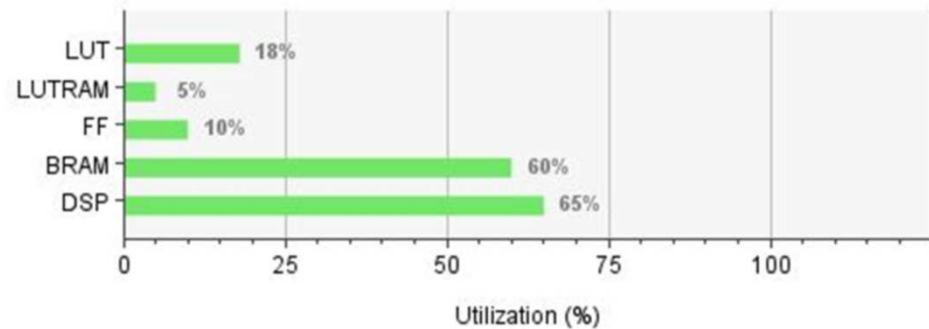
➤ 3.5 Vivado 합성 및 자원 사용 결과

✓ 세부 자원 사용량

보드 자원 한계 내에서 안정적으로 합성, 구현됨

Summary

Resource	Utilization	Available	Utilization %
LUT	9399	53200	17.67
LUTRAM	849	17400	4.88
FF	10634	106400	9.99
BRAM	84	140	60.00
DSP	144	220	65.45



➤ 3.5 Vivado 합성 및 자원 사용 결과

✓ 타이밍 체크

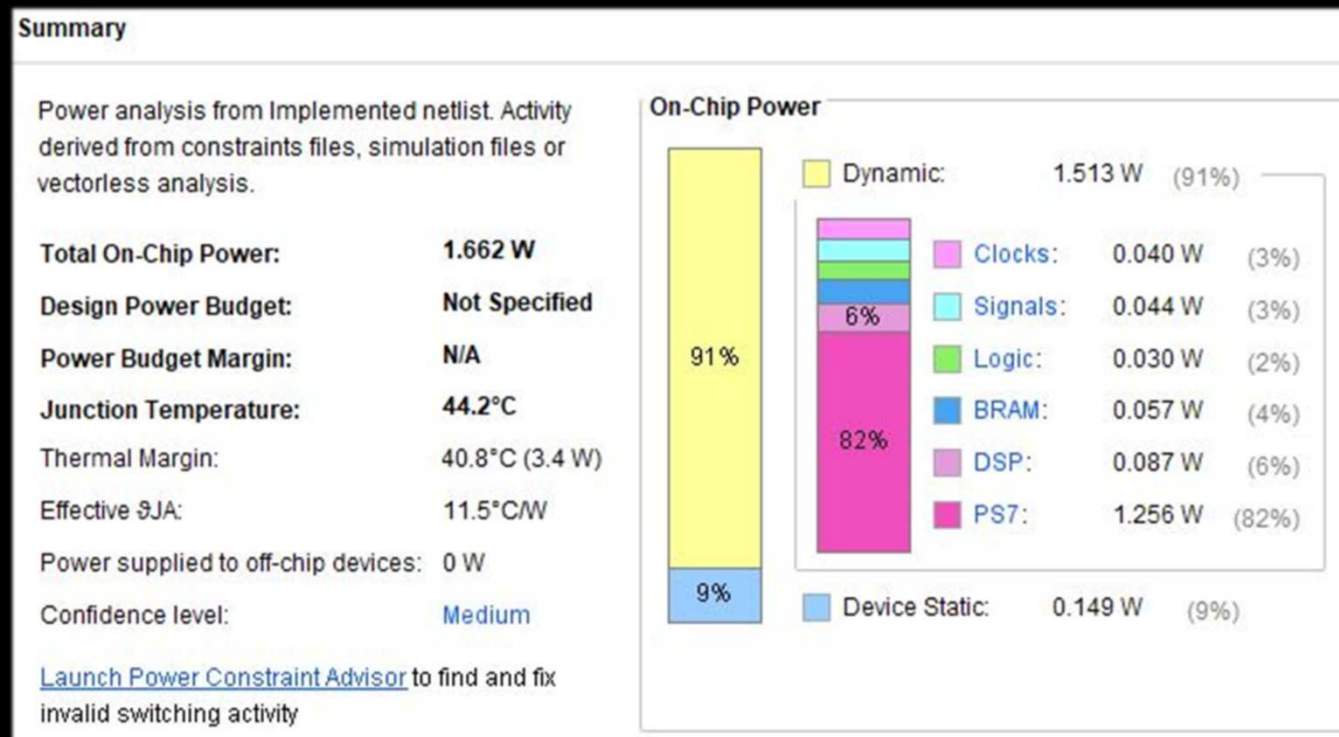
동작 주파수 100 MHz에서 Timing Violation 없이 동작

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 0.274 ns		Worst Hold Slack (WHS): 0.015 ns		Worst Pulse Width Slack (WPWS): 3.750 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 46754		Total Number of Endpoints: 46754		Total Number of Endpoints: 12474	
All user specified timing constraints are met.					

➤ 3.5 Vivado 합성 및 자원 사용 결과

✓ 전력 추정치

Total On-chip Power = Dynamic Power 1.513W + Static Power: 0.149W = 1.662W



➤ 4.1 실험 개요 & 4.2 실험 환경

✓ 실험 개요

설계한 컨볼루션 가속기는 Vivado + PYNQ 환경에서 4단계로 검증

1. RTL 시뮬레이션

Vivado 환경에서 Testbench를 통해 RTL 레벨에서의 정상 동작 검증

2. 기능 및 성능 검증

SoC 환경에서 랜덤 정수형 데이터를 기반으로 한 컨볼루션 가속 테스트
설계한 가속기의 정확성 검증 및 CPU(for-loop) 대비 연산 속도 향상 측정

3. 양자화 기반 Simple 모델 검증

임의의 CNN 모델을 구성하고, CIFAR-10 이미지 분류 추론 과정에서 FPGA로 컨볼루션 레이어 연산을 모두 대체
전 과정 PyTorch 추론 결과와 비교하여 분류 정확도 유지 및 처리속도 개선 확인

4. 양자화 기반 실제 모델 검증

실제 Torchvision 라이브러리에서 제공하는 VGG-16 모델을 바탕으로, 파인 튜닝을 거쳐 (3)과 동일한 방식의 실험을 진행

✓ 실험 환경

보드	Digilent Arty Z7-20 (Zynq XC7Z020-1CLG400C)
프로세서	ARM Cortex-A9 (Dual-core, 650 MHz)
소프트웨어	Vivado 2022.2 / PYNQ 2.7 (Jupyter Notebook)
통신 인터페이스	AXI4-Lite / AXI4-Stream
클럭 주파수	PL : 100 MHz

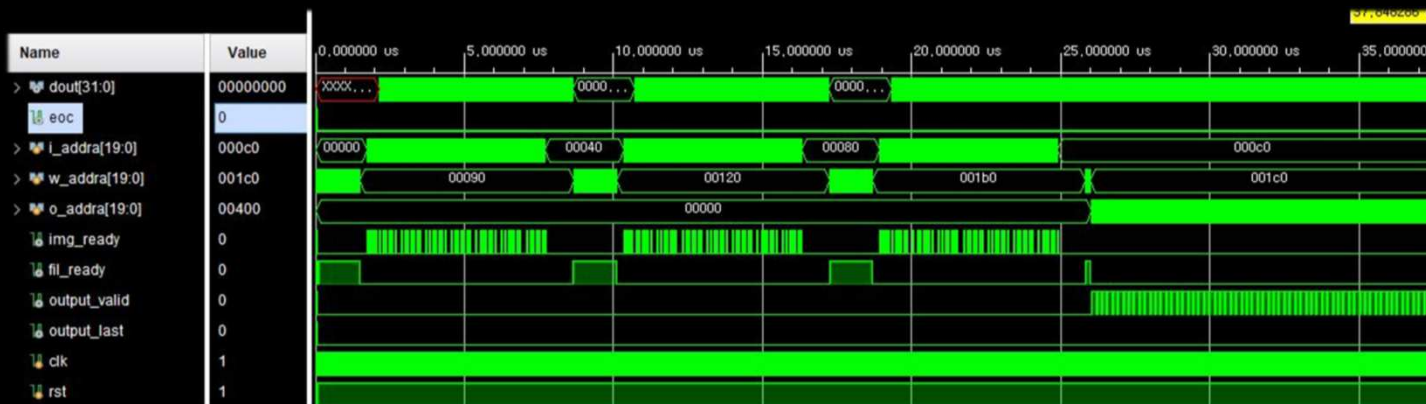
➤ 4.3 실험 1 – RTL 시뮬레이션 기초 동작 검증

✓ 목적

Vivado Simulator + Testbench를 이용해 컨볼루션 가속기의 기초 동작을 RTL 수준에서 검증 및 디버깅

✓ 구성 및 절차

1. Testbench 구성 - 설계한 모듈에 임의의 입력, 필터, 바이어스 데이터 인가
2. FSM 상태, 데이터 시프트 타이밍, 연산 출력 결과 등 관찰
3. RTL 코드 수정 시마다 반복 수행



```
#####
[0] Correct! Calculated : -2548, Golden : -2548
#####
[1] Correct! Calculated : -25498, Golden : -25498
#####
[2] Correct! Calculated : 12343, Golden : 12343
#####
[3] Correct! Calculated : -16758, Golden : -16758
#####
[4] Correct! Calculated : -17349, Golden : -17349
#####
[5] Correct! Calculated : 22456, Golden : 22456
#####
[6] Correct! Calculated : 16451, Golden : 16451
#####
[7] Correct! Calculated : -17578, Golden : -17578
#####
[8] Correct! Calculated : -81, Golden : -81
#####
[9] Correct! Calculated : 44718, Golden : 44718
#####
```

...

```
#####
[1021] Correct! Calculated : 1845, Golden : 1845
#####
[1022] Correct! Calculated : 41661, Golden : 41661
#####
[1023] Correct! Calculated : 22575, Golden : 22575
#####
The convolution module works well! ^_^
The total cycle is      3752
#####
```

➤ 4.4 실험 2 – 랜덤 정수 데이터 기반 기능 및 성능 검증

✓ 목적

FPGA에 구현된 컨볼루션 IP의 기능적 정확성 및 연산 성능 향상 효과 검증
임의의 정수형(INT8/INT32) 입력을 사용하여 CPU(for-loop) 대비 가속 효과 비교

✓ 구성 및 절차

1. Python에서 numpy로 임의의 정수형 입력/필터/바이어스 생성
2. 동일 데이터로 CPU vs FPGA 컨볼루션 연산 수행
3. 두 결과 일치 여부 검증 및 수행 시간 비교

=====

새로운 테스트 실행 시작

=====

[A] 테스트 파라미터 설정:

- image_width : 8
- input_channel : 1
- output_channel : 8
- kernel_size : 3
- stride : 1
- padding : 0

[A] 테스트 데이터 생성 중...

- 데이터 생성 완료.

[B] Golden Model (For-loop) 실행...

- Golden 결과 생성 완료 (소요 시간: 45.1524 ms)
- 예상 출력 크기: (288,)

[C] FPGA 가속기 실행...

- FPGA 연산 완료 (소요 시간: 1.5919 ms)

[D] 결과 검증...

성공: FPGA 결과가 Golden과 완벽히 일치합니다!

--- 결과 샘플 (앞 10개) ---

Index	Golden	FPGA
0	256154141	256154141
1	329528793	329528793
2	283217488	283217488
3	539979334	539979334
4	616468449	616468449
5	-435114858	-435114858
6	-859435547	-859435547
7	328981446	328981446
8	256156308	256156308
9	329505833	329505833

=====

테스트 실행 완료

=====

➤ 4.4 실험 2 – 랜덤 정수 데이터 기반 기능 및 성능 검증

✓ 결과 요약

입력크기	입력채널	출력채널	패딩	스트라이드	CPU(ms)	FPGA(ms)	속도 향상 비율	결과일치
8×8	1	8	0	1	45.409	1.6263	27.92x	○
16×16	3	16	1	3	172.325	1.6541	104.18x	○
32×32	16	32	2	5	2126.789	2.3093	920.97x	○

✓ 분석 및 결론

3번의 테스트에서 모두 FPGA 연산 결과와 CPU 연산 결과가 일치

병렬 구조와 파이프라인 처리로 인해, 입력 크기 및 채널 수가 증가할수록 FPGA의 연산 이점이 더욱 뚜렷

DMA 전송 오버헤드 존재에도 불구하고 FPGA 연산 성능 우세

→ 기능적 정확성 + 성능적 우수성 모두 입증

➤ 4.5 실험 3 – 양자화 기반 Simple VGG 모델 추론 검증

✓ 목적

학습된 Simple VGG 모델의 실제 추론 과정에 양자화 및 FPGA 가속기를 적용
전 과정 PyTorch 추론 결과와 비교하여 정확도 유지 및 처리속도 향상을 검증

✓ 구성 및 절차

1. 모델 학습 및 .pth파일 추출

CIFAR-10 데이터셋 기반 Simple VGG 모델 학습, 완료 후 .pth 가중치 파일 추출

2. 양자화 적용

컨볼루션 입력, 필터, 바이어스 → INT8, INT8, INT32 형식 변환

3. 하이브리드 추론 수행

(a) Baseline : 전 과정 PyTorch 부동소수점 연산으로 추론

(b) Hybrid : PyTorch 추론 중 컨볼루션 레이어만 FPGA로 우회해 가속

```
[추론 준비] 공용 DMA 버퍼 할당 시도...  
공용 DMA 버퍼 3개 할당 성공.
```

```
테스트셋 *1000*개 이미지에 대한 정확도 비교 시작...
```

[이미지 #0]	Time (CPU):	3405.02 ms		Time (FPGA):	275.65 ms
[이미지 #1]	Time (CPU):	3315.34 ms		Time (FPGA):	262.28 ms
[이미지 #2]	Time (CPU):	3316.26 ms		Time (FPGA):	263.57 ms
[이미지 #3]	Time (CPU):	3321.39 ms		Time (FPGA):	262.26 ms
[이미지 #4]	Time (CPU):	3316.81 ms		Time (FPGA):	263.62 ms
[이미지 #5]	Time (CPU):	3315.09 ms		Time (FPGA):	261.96 ms
[이미지 #6]	Time (CPU):	3314.53 ms		Time (FPGA):	263.94 ms
[이미지 #7]	Time (CPU):	3319.49 ms		Time (FPGA):	262.32 ms
[이미지 #8]	Time (CPU):	3319.39 ms		Time (FPGA):	263.57 ms
[이미지 #9]	Time (CPU):	3315.28 ms		Time (FPGA):	262.13 ms

...

```
=====
```

PYNQ 테스트 최종 정확도 비교 결과 (1000개 샘플)

```
=====
```

- [A] PyTorch CPU 추론 정확도:	80.70 % (807/1000)
- [B] PYNQ FPGA 가속 추론 정확도:	80.60 % (806/1000)

```
-----
```

- 정확도 차이 (A - B): +0.10 %

```
=====
```

PYNQ 테스트 최종 성능 비교 결과 (1000개 샘플)

```
=====
```

- [A] CPU 평균 추론 시간:	3314.74 ms/image
- [B] FPGA 평균 추론 시간:	260.89 ms/image

```
-----
```

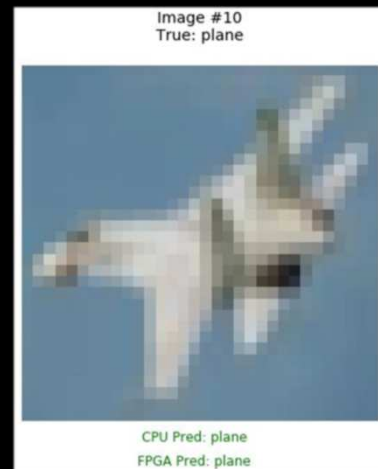
- FPGA 가속 성능: 12.71x (CPU 대비)

➤ 4.5 실험 3 – 양자화 기반 Simple VGG 모델 추론 검증

✓ 결과 요약

분류 정확도 + 평균 추론 시간(1,000개 이미지 평균) 비교

구분	추론 정확도(%)	평균 추론 시간(ms)	속도 향상 비율
PyTorch	80.7%	3314.74	-
Hybrid (Conv IP + PyTorch)	80.6%	260.89	12.71x



분류 성공



분류 실패

✓ 분석 및 결론

양자화 및 하이브리드 추론의 분류 정확도 손실 0.1%p 이내 → 거의 동일한 분류 정확도 유지

평균 추론 속도 약 12.71배 향상 → 컨볼루션 레이어 가속만으로도 CNN 전체 추론의 시간이 크게 감소함

실제 모델 추론 환경에서도 본 연구에서 제안한 CNN 컨볼루션 IP의 효율성과 실용성을 입증

➤ 4.6 실험 4 – 양자화 기반 VGG 16 모델 추론 검증

✓ 목적

학습된 VGG16 모델의 실제 추론 과정에 양자화 및 FPGA 가속기를 적용
전 과정 PyTorch 추론 결과와 비교하여 정확도 유지 및 처리속도 향상을 검증

✓ VGG 16 모델 구성 및 파인튜닝

1. Max Pooling 일부 제거(layer 3개)로 저해상도 이미지의 정보 손실 방지
2. Conv Layer를 동결(Freeze)하여 가중치 손상을 막고, 재설계된 분류기만 학습
3. VGG16의 경우 imagenet 이미지를 input으로 학습된 모델이기 때문에, 파인튜닝을 통해 CIFAR-10에 맞게 수정
4. 전이 학습(Transfer Learning): ImageNet Pre-trained 가중치를 적용하여 특징 추출 능력 확보

➤ 4.6 실험 4 – 양자화 기반 VGG 16 모델 추론 검증

✓ 테스트 이미지 설정

PYNQ CPU의 연산시간이 매우 오래 걸려 모든 이미지 테스트 불가

→ 임의의 테스트셋 이미지 100개에 대한 정확도만 비교

✓ 구성 및 절차

1. 모델 학습 및 .pth파일 추출

CIFAR-10 데이터셋 기반 Simple VGG 모델 학습, 완료 후 .pth 가중치 파일 추출

2. 양자화 적용

컨볼루션 입력, 필터, 바이어스 → INT8, INT8, INT32 형식 변환

3. 하이브리드 추론 수행

(a) Baseline : 전 과정 PyTorch 부동소수점 연산으로 추론

(b) Hybrid : PyTorch 추론 중 컨볼루션 레이어만 FPGA로 우회해 가속

[추론 준비] 공용 DMA 버퍼 할당 시도...
공용 DMA 버퍼 3개 할당 성공.

테스트셋 *100개 이미지에 대한 정확도 비교 시작...

[이미지 #0]	Time (CPU): 22099.34 ms		Time (FPGA): 1401.33 ms
[이미지 #1]	Time (CPU): 21850.25 ms		Time (FPGA): 1356.67 ms
[이미지 #2]	Time (CPU): 22683.89 ms		Time (FPGA): 1371.99 ms
[이미지 #3]	Time (CPU): 23293.70 ms		Time (FPGA): 1374.85 ms
[이미지 #4]	Time (CPU): 23449.30 ms		Time (FPGA): 1400.19 ms
[이미지 #5]	Time (CPU): 23450.62 ms		Time (FPGA): 1398.58 ms
[이미지 #6]	Time (CPU): 23450.37 ms		Time (FPGA): 1371.50 ms
[이미지 #7]	Time (CPU): 23447.85 ms		Time (FPGA): 1375.34 ms
[이미지 #8]	Time (CPU): 23468.82 ms		Time (FPGA): 1370.68 ms
[이미지 #9]	Time (CPU): 23448.46 ms		Time (FPGA): 1372.12 ms

PYNQ 테스트 최종 정확도 비교 결과 (100개 샘플)

- [A] PyTorch CPU 추론 정확도:	83.00 % (83/100)
- [B] PYNQ FPGA 가속 추론 정확도:	81.00 % (81/100)
- 정확도 차이 (A - B): +2.00 %	

PYNQ 테스트 최종 성능 비교 결과 (100개 샘플)

- [A] CPU 평균 추론 시간:	23223.78 ms/image
- [B] FPGA 평균 추론 시간:	1375.73 ms/image
- FPGA 가속 성능: 16.88x (CPU 대비)	

➤ 4.6 실험 4 – 양자화 기반 VGG 16 모델 추론 검증

✓ 결과 요약

분류 정확도 + 평균 추론 시간(100개 이미지 평균) 비교

구분	추론 정확도(%)	평균 추론 시간(ms)	속도 향상 비율
PyTorch	83%	23223.78	-
Hybrid (Conv IP + PyTorch)	81%	1375.73	16.88×



분류 성공



분류 실패

✓ 분석 및 결론

양자화 및 하이브리드 추론의 분류 정확도 손실 2%p 이내 → 분류 정확도 하락률 매우 미미함

평균 추론 속도 약 16.88배 향상 → 컨볼루션 레이어 가속만으로도 CNN 전체 추론의 시간이 크게 감소

실제 모델 추론 환경에서도 본 연구에서 제안한 CNN 컨볼루션 IP의 효율성과 실용성을 입증

➤ 5. 결론 - 연구 및 주요 설계 요약

✓ 연구 요약

Zynq-7000 SoC (Arty Z7-20) 기반 FPGA에서 CNN 컨볼루션 연산을 효율적으로 가속하기 위한 가속 IP를 설계, 구현
입력 크기, 채널 수, 패딩, 스트라이드 등 다양한 조건을 단일 하드웨어로 자동 대응할 수 있는 준-범용 구조 실현
정수형 양자화 기반 HW/SW 통합 구조로 소형 임베디드 환경에서도 실시간 CNN 추론 가능성을 검증

✓ 주요 설계 요약

3×3 필터 전용 구조 → CNN에서 가장 널리 사용되는 필터 크기로, 효율과 범용성의 Trade-off
Shift Buffer 기반 입력 윈도우 구성 → 입력 데이터의 최대 재사용으로 DRAM 접근 최소화
16개 병렬 MAC9 유닛 → 높은 연산 병렬성 확보 및 처리 속도 향상
BRAM-FIFO 혼용 구조 → 필터, 바이어스 저장과 다채널 출력 누적의 효율적 처리
체계적인 FSM → 입력-연산-출력 전 과정을 하드웨어 단독 자동 수행
PYNQ 기반 드라이버 + DMA 통신 → HW/SW 완전 통합 CNN 추론 시스템 구현

➤ 5. 결론 - 실험 결과 및 연구 의의

✓ 실험 결과 요약

실험	내용	결과
2	랜덤 정수 데이터 기반 기능 및 성능 검증	CPU 대비 최대 920x 속도 향상, 결과 완전 일치
3	양자화 기반 Simple VGG 모델 추론 검증	분류 정확도 손실 0.1%p 이내, 추론 속도 12.71x 향상
4	양자화 기반 VGG 16 모델 추론 검증	분류 정확도 손실 2.0%p 이내, 추론 속도 16.88x 향상

→ 정확도 유지 + 연산 효율 향상을 동시에 달성

✓ 연구 의의 및 향후 발전

향후 Batch Normalization, ReLU 등 추가 연산 모듈을 포함한 특정 CNN 가속기 설계 시
누구나 본 IP를 컨볼루션 연산 전용 모듈로서 인스턴스화하여 활용 가능

Zynq UltraScale+ 등 상위 FPGA 보드로 이식할 시 더 높은 동작 주파수 및 자원 활용을 통해 연산 성능 향상 가능

본 IP는 준-범용성을 필두로 다양한 CNN 네트워크 아키텍처에 적용 가능함에 따라,
향후 CNN 가속기 설계의 효율성과 접근성을 높이는 데 기여할 수 있을 것으로 기대

➤ Appendix

예) 입력 크기 4x4, 입력 채널1, 출력 채널 1, 패딩 1, 스트라이드 3

(0, 0)

처음 →

0	0	0	0	0	0
0	1	2	3	4	0
0	5	6	7	8	0
0	9	10	11	12	0
0	13	14	15	16	0
0	0	0	0	0	0

(5, 5) ← 마지막

입력 이미지 맵 예시
(실제로는 Shift buffer에 저장)

str_row_cnt / str_col_cnt

-	-	-	-	-	-
-	-	-	-	-	-
-	-	0/0	0/1	0/2	0/0
-	-	1/0	1/1	1/2	1/0
-	-	2/0	2/1	2/2	2/0
-	-	0/0	0/1	0/2	0/0

스트라이드 카운터
(0/0 인 경우에만 컨볼루션 진행)

(img_row_cnt, img_col_cnt) = (0, 0)

(str_row_cnt, str_col_cnt) = (-, -)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35

0

Shift buffer

처음

(img_row_cnt, img_col_cnt) = (0, 1)

(str_row_cnt, str_col_cnt) = (-, -)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35

0

0

(img_row_cnt, img_col_cnt) = (0, 2)

(str_row_cnt, str_col_cnt) = (-, -)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	← 0														
		0													

(img_row_cnt, img_col_cnt) = (0, 3)

(str_row_cnt, str_col_cnt) = (-, -)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	0				0										
		0													

연결된 위치
이미지 크기 + 2*패딩 - 1 = 5

```
(img_row_cnt, img_col_cnt) = (0, 4)
```

```
(str_row_cnt, str_col_cnt) = (-, -)
```

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	0			0 ← 0											
		0													

```
(img_row_cnt, img_col_cnt) = (0, 5)
```

```
(str_row_cnt, str_col_cnt) = (-, -)
```

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	0		0	0	0										
		0													

```
(str_row_cnt, str_col_cnt) = (-, -)
```

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	0	0	0	0	0										
		0													

```
(str_row_cnt, str_col_cnt) = (-, -)
```

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
	0														
0	0	0	0	0	0										
		1													

```
(str_row_cnt, str_col_cnt) = (-, -)
```

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	0														
0	1	0	0	0	0										
		2													

```
(str_row_cnt, str_col_cnt) = (-, -)
```

[illegible]

(img_row_cnt, img_col_cnt) = (2, 0)

(str_row_cnt, str_col_cnt) = (-, -)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	0	0	0	0	0										
4	0	0	1	2	3										
		0													

MAC9 유닛 내 윈도우

		0
		0
		0

(img_row_cnt, img_col_cnt) = (2, 1)

(str_row_cnt, str_col_cnt) = (-, -)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	0	0	0	0	0										
0	0	1	2	3	4										
		5													

	0	0
	0	1
	0	5

(img_row_cnt, img_col_cnt) = (2, 2)

(str_row_cnt, str_col_cnt) = (0, 0)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	1	0	0	0	0										
0	5	2	3	4	0										
		6													

0	← 0	0
0	← 1	2
0	← 5	6

(img_row_cnt, img_col_cnt) = (2, 3)

(str_row_cnt, str_col_cnt) = (0, 1)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
		0	0	0	0										
		3	4	0	0										
		7													

0	0	0
1	2	3
5	6	7

(img_row_cnt, img_col_cnt) = (2, 4)

(str_row_cnt, str_col_cnt) = (0, 2)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
2	3	0	0	0	1										
6	7	4	0	0	5										
		8													

0	0	0
2	3	4
6	7	8

(img_row_cnt, img_col_cnt) = (2, 5)

(str_row_cnt, str_col_cnt) = (0, 0)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
3	4	0	0	1	2										
7	8	0	0	5	6										
		0													

0	0	0
3	4	0
7	8	0

(img_row_cnt, img_col_cnt) = (3, 0)

(str_row_cnt, str_col_cnt) = (-, -)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
4	0	0	1	2	3										
8	0	0	5	6	7										
		0													

0	0	0
4	0	0
8	0	0

(img_row_cnt, img_col_cnt) = (3, 1)

(str_row_cnt, str_col_cnt) = (-, -)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	0	1	2	3	4										
0	0	5	6	7	8										
		9													

0	0	1
0	0	5
0	0	9

(img_row_cnt, img_col_cnt) = (3, 2)

(str_row_cnt, str_col_cnt) = (1, 0)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	5	2	3	4	0										
0	9	6	7	8	0										
		10													

0	1	2
0	5	6
0	9	10

(img_row_cnt, img_col_cnt) = (3, 3)

(str_row_cnt, str_col_cnt) = (1, 1)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
5	6	3	4	0	0										
9	10	7	8	0	0										
		11													

1	2	3
5	6	7
9	10	11

(img_row_cnt, img_col_cnt) = (3, 4)

(str_row_cnt, str_col_cnt) = (1, 2)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
6	7	4	0	0	5										
10	11	8	0	0	9										
		12													

2	3	4
6	7	8
10	11	12

(img_row_cnt, img_col_cnt) = (3, 5)

(str_row_cnt, str_col_cnt) = (1, 0)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
7	8	0	0	5	6										
11	12	0	0	9	10										
		0													

3	4	0
7	8	0
11	12	0

(img_row_cnt, img_col_cnt) = (4, 0)

(str_row_cnt, str_col_cnt) = (-, -)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
8	0	0	5	6	7										
12	0	0	9	10	11										
		0													

4	0	0
8	0	0
12	0	0

(img_row_cnt, img_col_cnt) = (4, 1)

(str_row_cnt, str_col_cnt) = (-, -)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	0	5	6	7	8										
0	0	9	10	11	12										
		13													

0	0	5
0	0	9
0	0	13

(img_row_cnt, img_col_cnt) = (4, 2)

(str_row_cnt, str_col_cnt) = (2, 0)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	9	6	7	8	0										
0	13	10	11	12	0										
		14													

0	5	6
0	9	10
0	13	14

(img_row_cnt, img_col_cnt) = (4, 3)

(str_row_cnt, str_col_cnt) = (2, 1)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
		7	8	0	0										
9	10														
13	14	11	12	0	0										
		15													

5	6	7
9	10	11
13	14	15

(img_row_cnt, img_col_cnt) = (4, 4)

(str_row_cnt, str_col_cnt) = (2, 2)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
10	11	8	0	0	9										
14	15	12	0	0	13										
		16													

6	7	8
10	11	12
14	15	16

(img_row_cnt, img_col_cnt) = (4, 5)

(str_row_cnt, str_col_cnt) = (2, 0)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
11	12	0	0	9	10										
15	16	0	0	13	14										
		0													

7	8	0
11	12	0
15	16	0

(img_row_cnt, img_col_cnt) = (5, 0)

(str_row_cnt, str_col_cnt) = (-, -)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
12	0	0	9	10	11										
16	0	0	13	14	15										
		0													

8	0	0
12	0	0
16	0	0

(img_row_cnt, img_col_cnt) = (5, 1)

(str_row_cnt, str_col_cnt) = (-, -)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	0	9	10	11	12										
0	0	13	14	15	16										
		0													

0	0	9
0	0	13
0	0	0

(img_row_cnt, img_col_cnt) = (5, 2)

(str_row_cnt, str_col_cnt) = (0, 0)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
0	13	10	11	12	0										
0	0	14	15	16	0										
		0													

0	9	10
0	13	14
0	0	0

(img_row_cnt, img_col_cnt) = (5, 3)

(str_row_cnt, str_col_cnt) = (0, 1)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
13	14	11	12	0	0										
0	0	15	16	0	0										
		0													

9	10	11
13	14	15
0	0	0

(img_row_cnt, img_col_cnt) = (5, 4)

(str_row_cnt, str_col_cnt) = (0, 2)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
14	15	12	0	0	13										
0	0	16	0	0	0										
		0													

10	11	12
14	15	16
0	0	0

(img_row_cnt, img_col_cnt) = (5, 5)

(str_row_cnt, str_col_cnt) = (0, 0)

0	1	2	3	4	5	6	7	8	...	30	31	32	33	34	35
15	16	0	0	13	14										
0	0	0	0	0	0										
		0													

11	12	0
15	16	0
0	0	0

해당 알고리즘을 통해 어떠한 입력 크기, 패딩, 스트라이드 값에도 대응 가능

예) 입력 채널이 여러 개라면?

0	0	0	0	0	0
0	1	2	3	4	0
0	5	6	7	8	0
0	9	10	11	12	0
0	13	14	15	16	0
0	0	0	0	0	0

1	-1	1
1	-1	1
1	-1	1

1	-1	1
1	-1	1
1	-1	1

1	-1	1
1	-1	1
1	-1	1

0	0	0	0	0	0
0	1	2	3	4	0
0	5	6	7	8	0
0	9	10	11	12	0
0	13	14	15	16	0
0	0	0	0	0	0

1	-1	1
1	-1	1
1	-1	1

1	-1	1
1	-1	1
1	-1	1

1	-1	1
1	-1	1
1	-1	1



한 입력 채널씩 컨볼루션을 반복 후 누적
x 3

예) 출력 채널(필터)이 여러 개라면?

0	0	0	0	0	0
0	1	2	3	4	0
0	5	6	7	8	0
0	9	10	11	12	0
0	13	14	15	16	0
0	0	0	0	0	0

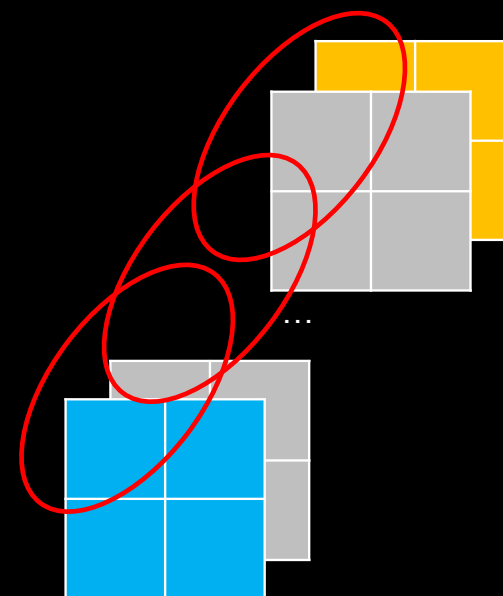
1	-1	1
1	-1	1
1	-1	1

1	-1	1
1	-1	1
1	-1	1

...

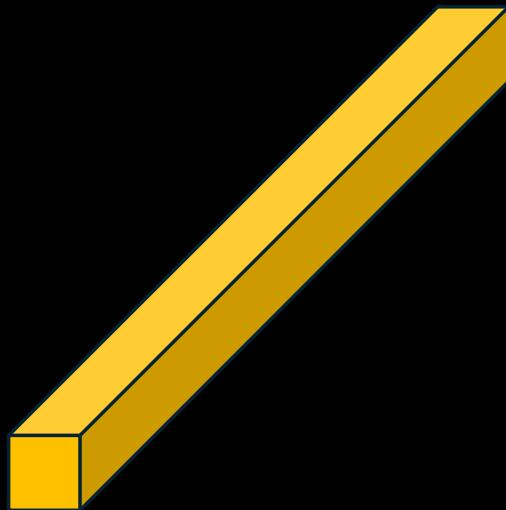
1	-1	1
1	-1	1
1	-1	1

1	-1	1
1	-1	1
1	-1	1



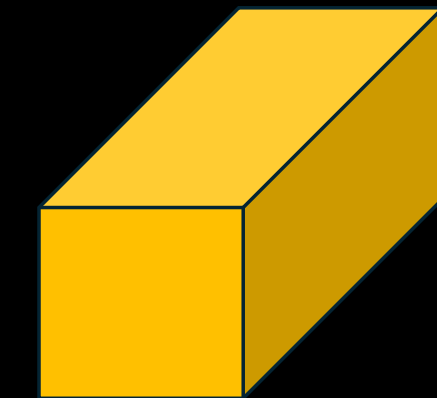
최대 16개 필터씩 분할, 파이프라인으로 처리

예) 한 입력 채널의 컨볼루션 결과를 저장할 때, FIFO의 메리트



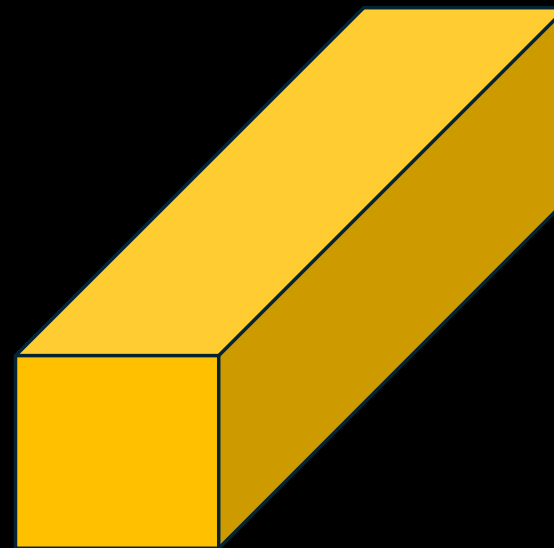
512x4x4

이미지 크기가 4x4
출력 채널이 512개인 경우



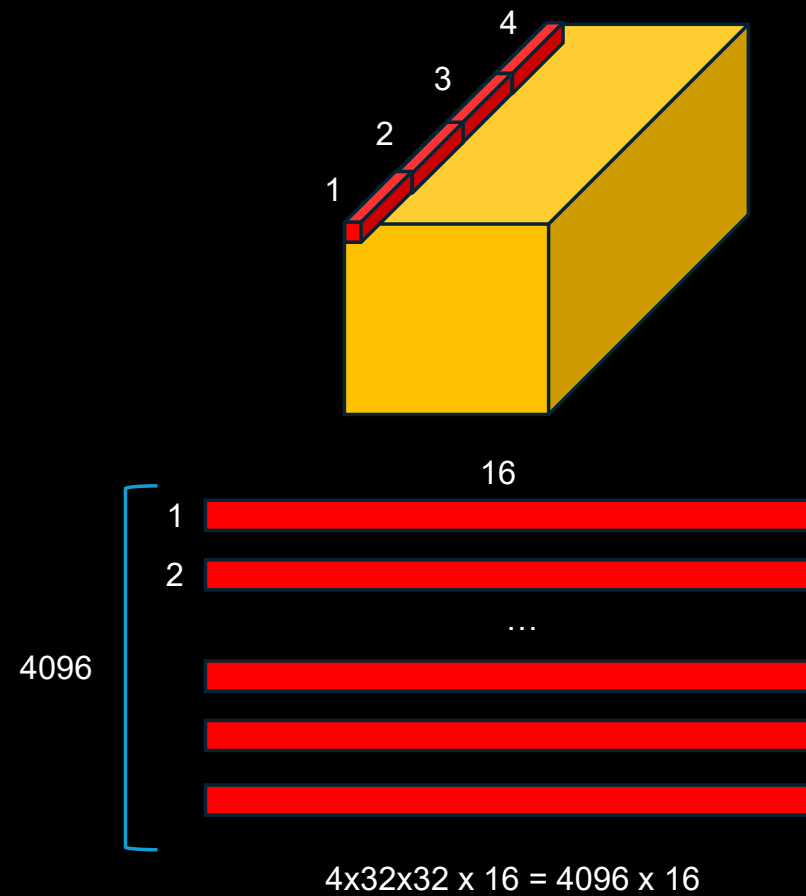
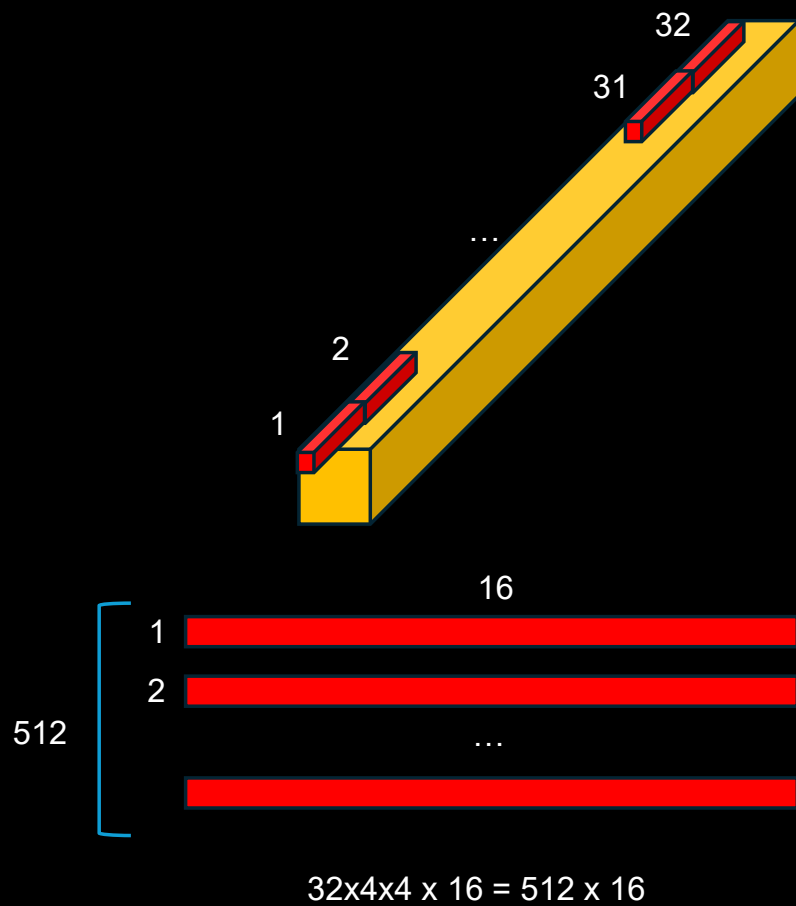
64x32x32

이미지 크기가 32x32
출력 채널이 64개인 경우



512x32x32

두 경우를 모두 담기 위해서
RAM은 512x32x32 만큼 필요



1. 4096 x 16 만큼의 FIFO만으로 두 경우 모두 담기 가능
2. 다른 입력 채널에 대해서도 동일한 순서로 연산되므로, 연산 결과를 FIFO의 top과 더해 다시 FIFO에 삽입하는 식으로 쉽게 다채널 누적 가능

Thank You